Reddit Tracker — API Call Architecture

How new.json and info.json calls are batched · Rate limit analysis · Capacity planning

---

1. Overview

Every 60 seconds, the poll worker runs a single global job that covers **all active subreddits at once**. It makes two types of Reddit API calls per cycle:

- **new.json calls** — fetch recently posted content across subreddits
- **info.json calls** — refresh engagement scores (upvotes + comments) for all Stack 1 and Stack 2 posts

Reddit's public JSON API allows approximately **60 requests per minute** (1 req/sec). Staying under this limit is the core constraint for capacity planning.

MASTER FORMULA

```
Total calls/min = ceil(N / 5) + ceil(N × P / 100)

where  N = number of active subreddits
      P = average number of active Stack 1 + Stack 2 posts per subreddit
```

Why these numbers? Each new.json batch groups 5 subreddits (CHUNK_SIZE = 5) and fetches 100 posts total (~20/subreddit). Each info.json batch refreshes up to 100 post IDs in a single call. Stack 1 and Stack 2 posts are both refreshed so engagement is current at every transition check.

---

2. new.json Batching

How it works

Instead of one call per subreddit, all subreddits are grouped into batches of 5 using Reddit's multi-subreddit syntax. Each batch is one HTTP request:

```
GET /r/sub1+sub2+sub3+sub4+sub5/new.json?limit=100
```

Reddit returns up to 100 posts from the combined feed of all 5 subreddits — approximately **~20 posts per subreddit**. The distribution is based on recency (newest first across all 5), so an unusually active subreddit may take more of the 100 slots.

Call count formula

```
new.json calls = ceil(N / 5)

Examples:   5 subs → 1 call   10 subs → 2 calls   34 subs → 7 calls  (6 batches of 5 + 1 batch of 4)   50 subs → 10 calls  100 subs → 20 calls
```

What happens when you add a new subreddit?

No manual action is required. The poll worker queries all active subreddits from the database at the start of every 60-second cycle. A newly added subreddit is automatically included in the next chunked batch — it simply becomes part of the pool. There is no per-subreddit scheduling, no queue entry to create, and no restart needed.

> **Example:** You have 34 subreddits (7 new.json calls). You add 1 more. On the very next poll cycle (within 60 seconds), it runs with 35 subreddits → still 7 calls (ceil(35/5) = 7). No change in call count.

---

3. info.json Batching

How it works

At each poll cycle, the worker fetches *all* active Stack 1 and Stack 2 posts from *all* subreddits in a single database query. These IDs are combined into one flat list and passed to Reddit's global info endpoint in chunks of 100:

```
GET /api/info.json?id=t3_abc,t3_def,t3_ghi,... (up to 100 IDs per call)
```

This endpoint accepts post IDs from any subreddit — no subreddit path needed. The response contains the latest upvote and comment counts, which are written back to the database **before** stack transition checks run.

Which stacks are refreshed, and why?

- **Stack 1** — refreshed ✅. The s1MinEng check happens at transition time (after s1MinAge elapsed), not at insertion time. The post needs current engagement at that moment to be evaluated fairly.
- **Stack 2** — refreshed ✅. Needs live engagement to calculate growth % during the eval window. Without fresh data, growth comparison would use stale values.
- **Stack 3** — not refreshed. Terminal (already alerted). Waiting for 4-hour expiry. No further evaluation needed.

Call count formula

```
info.json calls = ceil(N × P / 100)

where P = average active Stack 1 + Stack 2 posts per subreddit
```

Example with P = 12 (low estimate: avg 7 S1 + 5 S2 posts/subreddit)

Subreddits (N)	Total S1+S2 posts (N×12)	info.json calls
10	120	2
34 ← current	408	5
50	600	6
100	1,200	12
185	2,220	23

Example with P = 17 (high estimate: avg 10 S1 + 7 S2 posts/subreddit)

Subreddits (N)	Total S1+S2 posts (N×17)	info.json calls
10	170	2
34 ← current	578	6
50	850	9
100	1,700	17
160	2,720	28

4. Total Calls Per Minute

With P = 12 (low: 7 S1 + 5 S2 posts/sub)

Subreddits	new.json	info.json	Total
34 ← current	7	5	12
30	6	4	**10**
60	12	8	**20**
91	19	11	**30**
125	25	15	**40**
155	31	19	**50**
185 ← max	**37**	**23**	**60 ⚠ limit**

With P = 17 (high: 10 S1 + 7 S2 posts/sub)

Subreddits	new.json	info.json	Total
34 ← current	7	6	13
25	5	5	**10**
52	11	9	**20**
78	16	14	**30**
104	21	18	**≈39** *
131	27	23	**50**
160 ← max	**32**	**28**	**60 ⚠ limit**

* Exact 40 not achievable with integer N at P=17 due to ceiling effects; 39 (N=104) or 41 (N=106) are nearest.

5. Formula Verification

Your proposed formula is correct:

```
ceil(N / 5) + ceil(N × P / 100) < 60
```

The approximate closed-form version (ignoring ceiling effects for large N) is:

```
N × (20 + P) / 100 < 60

→ N < 6000 / (20 + P)
```

Avg S1+S2 posts / sub (P)	Max subreddits (approx)	Exact max (formula check)
5	6000 / 25 = 240	240: ceil(240/5)+ceil(240×5/100) = 48+12 = 60 ✓
10	6000 / 30 = 200	200: 40+20 = 60 ✓
12 ← low estimate	**6000 / 32 = 187.5**	**185: ceil(185/5)+ceil(185×12/100) = 37+23 = 60 ✓**
15	6000 / 35 ≈ 171	170: ceil(170/5)+ceil(170×15/100) = 34+26 = 60 ✓
17 ← high estimate	**6000 / 37 ≈ 162**	**160: ceil(160/5)+ceil(160×17/100) = 32+28 = 60 ✓**
20	6000 / 40 = 150	150: 30+30 = 60 ✓
30	6000 / 50 = 120	120: 24+36 = 60 ✓
50	6000 / 70 ≈ 85	85: ceil(85/5)+ceil(85×50/100) = 17+43 = 60 ✓

6. Extreme Cases & Edge Conditions

Case 1 — Very few subreddits (1–5)

N	new.json	info.json (P=12)	Total	Notes
1	1	1	2	Minimum possible: 2 calls/min
5	1	1	2	5 subs still fit in 1 batch
6	2	1	3	Splits into batch of 5 + batch of 1

Case 2 — Subreddit added mid-cycle

If a subreddit is added while a poll cycle is already running, it is picked up on the **next cycle** (within 60 seconds). There is no partial-cycle inclusion. The DB query happens at the very start of each job.

Case 3 — Subreddit removed

Removing a subreddit sets `isActive = false` in the DB. On the next cycle, the worker's query excludes it — no queue cleanup needed. Its existing posts in S1/S2/S3 remain in the DB but will no longer be evaluated or refreshed.

Case 4 — Many active posts (high P)

Watch out: If a subreddit has a long eval window (e.g., s2EvalEnd = 20min) and posts frequently, P can spike. At P=50 with 34 subreddits:
ceil(34/5) + ceil(34×50/100) = 7 + 17 = **24 calls/min** — still well within limit.

At P=50, the hard cap is ~85 subreddits before hitting 60 req/min.

Case 5 — One subreddit is dead / private (404)

If a subreddit returns a non-200 response, that chunk is skipped silently (`if (!res.ok) continue`). Other chunks in the same cycle still run. The subreddit stays in the active list — you would need to manually remove it via the UI.

Case 6 — Rate limited (429)

If Reddit returns 429, the entire poll job throws and BullMQ retries it with exponential backoff (5s → 10s → 20s, up to 3 attempts). This means a rate-limited cycle is skipped rather than partially executed.

---

7. Current System State (34 Subreddits)

7
NEW.JSON CALLS / MIN
ceil(34 / 5) = 7
Batches: 6×5 subs + 1×4 subs
~20 posts per subreddit per batch

5 – 6
INFO.JSON CALLS / MIN
P=12 → ceil(408/100) = 5 calls
P=17 → ceil(578/100) = 6 calls
Stack 1 + Stack 2 posts refreshed

Metric	P = 12 (low: 7 S1 + 5 S2)	P = 17 (high: 10 S1 + 7 S2)
Total calls/min	12	13
% of rate limit used	20%	22%
Remaining headroom	48 calls/min	47 calls/min
Can add how many more subs?	~151 more (total 185)	~126 more (total 160)

Conclusion: At 34 subreddits you are using only ~20–22% of the rate limit. You have enormous headroom. Even at P=17 (high-traffic estimate), you can scale to **160 subreddits** before approaching Reddit's 60 req/min ceiling. At the low estimate (P=12), capacity extends to **185 subreddits**.